TemporalDifference

Implementazione "from scratch" e analisi sperimentale
di algoritmi di Reinforcement Learning su CartPole, Acrobot e Pong

Federico Sabbadini
M.Sc. Computer Engineering — Università degli Studi di Brescia
Corso di Reinforcement Learning, A.A. 2025/26

18 luglio 2026

Sommario

Ho implementato da zero i quattro algoritmi tabellari richiesti (Q-Learning, SARSA, Monte Carlo e TD(0)) e un agente Deep Q-Network in PyTorch, applicandoli a tre ambienti Gymnasium: CartPole, Acrobot e Pong. Coerentemente con la consegna, l'obiettivo non è la performance assoluta ma la *comprensione* del comportamento degli algoritmi e l'analisi critica dei risultati; ogni risultato è mediato su 3 seed. Il lavoro comprende le analisi di sensibilità a ε, α e γ, il confronto SARSA vs Q-Learning e MC vs TD(0), tre casi di fallimento espliciti e l'ablazione del DQN (target network ed experience replay) con il bonus Double DQN.

---

1 Introduzione

1.1 Obiettivi

Il progetto implementa *da zero* i principali algoritmi di Reinforcement Learning e li confronta su tre ambienti di difficoltà crescente. Come richiesto dalla consegna, l'enfasi è sulla **comprensione** e sull'**analisi sperimentale critica**, piuttosto che sulla performance assoluta: diversi risultati sono volutamente sub-ottimali e vengono analizzati come tali, compresi i casi di fallimento e i risultati che hanno smentito le ipotesi da cui ero partito durante i primi esperimenti.

1.2 Ambienti e algoritmi

Ho lavorato su tre ambienti Gymnasium: **CartPole-v1** (comune; stato 4-D continuo, 2 azioni, reward +1/passo, denso), **Acrobot-v1** (assegnato; 6-D continuo, 3 azioni, reward -1/passo fino all'obiettivo, sparso) e **ALE/Pong-v5** (assegnato; Atari da pixel, tramite l'Arcade Learning Environment). La progressione non è casuale: CartPole permette di verificare che gli algoritmi siano corretti, Acrobot li mette in crisi su un problema più complesso, e Pong mostra dove il passaggio a un approssimatore di funzione diventa necessario.

Su CartPole e Acrobot impiego quattro algoritmi tabellari scritti a mano: due di *controllo* (Q-Learning, SARSA) e due di *predizione* (Monte Carlo, TD(0)). Poiché entrambi gli ambienti presentano stato continuo, ho introdotto una **discretizzazione** per rendere praticabile la Q-table. Per Pong ho implementato un DQN in PyTorch (Mnih et al. 2015) con experience replay e target network, oltre al bonus Double DQN (van Hasselt et al. 2016).

MC e TD(0), in quanto algoritmi di predizione, valutano una policy *fissa* senza apprenderla: ho quindi scelto di addestrare prima Q-Learning, estrarne la policy greedy e valutare *quella*, affiancandole una policy casuale come riferimento (Sez. 3.3). La scelta non è neutra, poiché il divario tra MC e TD(0) in termini di stime dipende proprio dalla policy che si valuta e dalla lunghezza degli episodi che essa genera.

2 Fondamenti teorici

Si riportano solo i concetti necessari a comprendere le implementazioni, con la notazione del corso (Sutton & Barto, S&B nel seguito).

2.1 MDP, valore e policy

Un problema di RL è modellato come un *Markov Decision Process* (MDP): ad ogni *timestep* l'agente osserva uno *stato* s, sceglie un'*azione* a secondo una *policy* $\pi(a|s)$, riceve un *reward* R e transita nello stato successivo s'. L'obiettivo è massimizzare il *return* atteso, definito come il reward cumulato scontato

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \tag{1}$$

dove $\gamma \in [0, 1]$ è il fattore di sconto ($\gamma \to 0$ rende l'agente "miope", $\gamma \to 1$ "lungimirante"). A partire dal return si definiscono la *state-value* e la *action-value* di una policy π:

$$v_\pi(s) = \mathbb{E}[G_t \mid S_t = s], \qquad q_\pi(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a]. \tag{2}$$

2.2 Predizione vs controllo

È fondamentale distinguere i due compiti, poiché la consegna li richiede entrambi:

- **Predizione**: stimare v_π per una policy π *fissata*. Monte Carlo e TD(0) sono algoritmi di predizione.
- **Controllo**: trovare una policy migliore. Q-Learning e SARSA sono algoritmi di controllo.

2.3 Monte Carlo (MC) e TD(0)

Entrambi stimano v_π campionando l'equazione di Bellman, ma differiscono nel target utilizzato:

$$\text{MC:} \quad v(S_t) \leftarrow v(S_t) + \alpha \, [\, G_t - v(S_t) \,] \tag{3}$$

$$\text{TD(0):} \quad v(S_t) \leftarrow v(S_t) + \alpha \, \Big[\, \underbrace{R_{t+1} + \gamma \, v(S_{t+1})}_{\text{target}} - v(S_t) \,\Big] \tag{4}$$

MC impiega il return *effettivo* G_t: la stima che ne risulta è non distorta (*unbiased*) ma ad alta varianza, e richiede che l'episodio sia terminato prima di poter aggiornare. TD(0) impiega invece il *bootstrap* $R_{t+1} + \gamma \, v(S_{t+1})$ come target: la stima è distorta (poiché usa a sua volta una stima), ma presenta varianza più bassa e può essere aggiornata online ad ogni passo. La quantità $\delta_t = R_{t+1} + \gamma \, v(S_{t+1}) - v(S_t)$ prende il nome di *TD error*.

L'origine del compromesso è intuitiva. Il return G_t di MC dipende da *molte* azioni, transizioni e reward casuali lungo l'intera traiettoria, la cui variabilità si accumula tanto più l'episodio è lungo: per questo su traiettorie estese la stima MC oscilla molto più che su episodi corti. Il target di TD(0) dipende invece da *una* sola transizione casuale e ha quindi varianza minore; tuttavia, finché la stima $v(S_{t+1})$ è ancora inaccurata, essa trascina con sé il target, dando origine alla distorsione. Nel caso tabellare entrambi convergono a v_π; la velocità con cui il bias di TD si riassorbe dipende da quanto lo stato è "lontano" dal reward, ed è ciò che l'esperimento di predizione (Sez. 5.4) rende osservabile.

2.4 SARSA (on-policy) e Q-Learning (off-policy)

Sono l'aggiornamento TD applicato alle action-value q (S&B §6.4–6.5):

$$\text{SARSA:} \quad q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha \, [\, R_{t+1} + \gamma \, q(S_{t+1}, A_{t+1}) - q(S_t, A_t) \,] \tag{5}$$

$$\text{Q-Learning:} \quad q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha \, [\, R_{t+1} + \gamma \max_{a'} q(S_{t+1}, a') - q(S_t, A_t) \,] \tag{6}$$

SARSA è *on-policy*: il target impiega l'azione A_{t+1} effettivamente scelta dalla policy ε-greedy, tenendo quindi conto del rischio introdotto dall'esplorazione (la quintupla $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ dà il nome all'algoritmo). Q-Learning è invece *off-policy*: il target usa il max sulle azioni possibili, stimando così il valore della policy greedy indipendentemente dalla strategia esplorativa adottata, e converge a q_* purché ogni azione venga scelta infinite volte.

La conseguenza pratica è che i due algoritmi apprendono policy *qualitativamente* diverse quando l'esplorazione comporta un rischio rilevante. L'esempio canonico è il CliffWalking (S&B Fig. 6.5): il percorso ottimo passa sul bordo di un dirupo, e una singola azione esplorativa lungo quel bordo fa cadere l'agente. Q-Learning, il cui target assume comportamento greedy, apprende il percorso ottimo *sul bordo* e paga care le azioni esplorative; SARSA, che nel target include l'azione ε-greedy che realmente eseguirà, apprende un percorso più interno e "prudente", che rende di più *finché si continua a esplorare*. Questa distinzione, in forma attenuata, spiega il confronto SARSA vs Q-Learning su CartPole (Sez. 5.1).

2.5 Esplorazione: ε-greedy

La policy ε-greedy sceglie un'azione casuale con probabilità ε (*exploration*) e l'azione greedy $\arg\max_a q(s,a)$ altrimenti (*exploitation*); con $|\mathcal{A}|$ azioni,

$$\pi(a|s) = \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}| & \text{se } a = \arg\max_b q(s,b) \\ \varepsilon/|\mathcal{A}| & \text{altrimenti.} \end{cases} \tag{7}$$

Il valore di ε può essere tenuto *costante* oppure fatto *decrescere* nel tempo (*annealing*): si parte da un valore alto per favorire l'esplorazione e lo si riduce gradualmente per permettere all'agente di sfruttare le conoscenze acquisite. Nel progetto utilizzo entrambe le varianti e le confronto (Sez. 5.3, 5.7); il DQN impiega sempre l'annealing. Per il decadimento ho adottato un'unica implementazione lineare condivisa tra algoritmi tabellari e DQN:

$$\text{frac} = \min(1, \text{current}/\text{horizon}), \qquad \varepsilon = \text{start} + (\text{end} - \text{start}) \cdot \text{frac}. \tag{8}$$

2.6 Deep Q-Network (DQN)

Quando lo spazio degli stati è troppo grande per una tabella (come nel caso dei pixel di Pong), la action-value viene approssimata da una rete neurale $Q(s,a;\theta)$ di parametri θ (Mnih et al. 2015). Due componenti stabilizzano l'addestramento:

- **Experience Replay**: le transizioni vengono salvate in un buffer e campionate in mini-batch casuali, rompendo così la correlazione temporale tra campioni consecutivi.
- **Target Network**: una copia θ^- della rete, aggiornata di rado, fornisce target stabili $y = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a'; \theta^-)$, riducendo il rischio di divergenza dell'addestramento (*soft divergence*).

Il **Double DQN** (van Hasselt et al. 2016) parte dall'osservazione che $\max_{a'} q(S_{t+1}, a') = q\big(S_{t+1}, \arg\max_{a'} q(S_{t+1}, a')\big)$ impiega gli *stessi* valori sia per selezionare sia per valutare l'azione migliore, il che su stime rumorose causa una *sovrastima* sistematica (*maximization bias*, S&B §6.7). Per risolvere il problema, il Double DQN disaccoppia selezione e valutazione: $y = R_{t+1} + \gamma\, Q\big(S_{t+1}, \arg\max_{a'} Q(S_{t+1}, a'; \theta); \theta^-\big)$, utilizzando la rete online per scegliere l'azione e la target network per valutarla.

3 Metodologia e implementazione

3.1 Organizzazione del codice

Il codice si trova nel pacchetto `rl_lab`: gli algoritmi sono implementati come funzioni pure (ambiente in ingresso, risultati in uscita), gli script orchestrano gli esperimenti, il plotting è separato dai loop di training e gli iperparametri risiedono in `configs.py`. Le componenti principali sono `algorithms/tabular/` (i quattro algoritmi tabellari), `algorithms/deep/` (rete Nature-CNN, replay buffer, loop DQN con i flag di ablazione e Double DQN), `envs/` (discretizzatore e preprocessing Atari), `experiments/runner.py` (esecuzione multi-seed: ogni esperimento transita per questo modulo, rendendo impossibile produrre accidentalmente un risultato a seed singolo) e `utils/` (seeding e plotting).

Tabella 1: Caratteristiche degli ambienti e discretizzazione adottata.

Ambiente	Stato	Azioni	Reward	Discretizzazione
CartPole	4-D cont.	2	+1/passo (denso)	10 bin/dim $\rightarrow 10^4$ stati
Acrobot	6-D cont.	3	−1/passo (sparso)	6 bin/dim $\rightarrow$ 46 656 stati
Pong	210×160 RGB	6	±1 (punto)	— (pixel, DQN)

3.2 Discretizzazione: le scelte che contano

Il wrapper `DiscretizedEnv` clippa ogni dimensione dell'osservazione in un intervallo configurato, la scala linearmente e la tronca a un indice intero; il risultato è una tupla utilizzata come chiave della Q-table (*state aggregation*). Due scelte influiscono in modo significativo sui risultati e vengono dichiarate di seguito.

Intervalli di clipping. Su CartPole il clipping sulla posizione del carrello e sull'angolo dell'asta avviene ai limiti di **terminazione** (±2.4 e ±0.21 rad), e non a quelli dello spazio di osservazione (±4.8, ±0.418): oltre le soglie di terminazione l'episodio è già concluso, dunque usare i limiti più larghi significherebbe sprecare metà dei bin di posizione e angolo su stati che non possono mai verificarsi. Le velocità, prive di un limite ufficiale, sono clippate a ±3.0, un valore entro cui restano comodamente durante il bilanciamento. Su Acrobot i limiti esistono nel sorgente Gymnasium (4π e 9π per le due velocità angolari) e vengono riutilizzati esattamente.

Numero di bin. Si tratta del parametro che ho preferito non assumere a priori: la Sez. 5.6 lo misura direttamente e ne valuta l'impatto sulla performance. Per CartPole sono impiegati 10 bin per dimensione, per Acrobot 6.

3.3 Quale policy valutano MC e TD(0)?

Per ottenere una policy fissa da valutare, addestro Q-Learning e ne estraggo la policy greedy $\pi(s) = \arg\max_a Q(s, a)$; entrambi gli stimatori vengono applicati a questa policy, e come riferimento anche a una policy *casuale*. Il doppio caso non è ridondante e rappresenta anzi la parte più interessante del risultato (Sez. 5.4): la policy appresa genera episodi lunghi ($\approx$ 330 passi), quella casuale episodi corti ($\approx$ 22).

Per confrontare gli stimatori traccio la stima $V(s_{\text{ref}})$ di un singolo stato di riferimento, episodio per episodio. Come s_{ref} ho scelto lo **stato di reset dell'ambiente**, che è sempre raggiungibile e viene visitato all'inizio di ogni episodio, garantendo così che entrambi gli stimatori lo aggiornino costantemente. La mia prima scelta era stata il "bin centrale" di ogni dimensione, ma su Acrobot questo produce uno stato *fisicamente impossibile*, poiché l'osservazione codifica gli angoli come $[\cos\theta, \sin\theta]$, che non possono essere entrambi ≈ 0.

3.4 DQN, preprocessing Atari e riproducibilità

La rete adottata è la Nature-CNN (Mnih et al. 2015): tre layer convoluzionali seguiti da due fully connected ($3136 \rightarrow 512 \rightarrow n_{\text{azioni}}$), che restituiscono $Q(s, \cdot)$ per tutte le azioni in un singolo forward pass.

Il preprocessing Atari segue lo standard di Mnih et al.: no-op iniziali casuali, frame-skip 4 con massimo pixel-wise sugli ultimi due frame (necessario perché Atari fa lampeggiare gli sprite e la pallina può scomparire a frame alterni), conversione a scala di grigi, ridimensionamento a 84×84 e stack dei 4 frame più recenti, in modo che la rete possa inferire la direzione della pallina. I frame vengono mantenuti in formato `uint8` nel buffer e normalizzati in $[0, 1]$ solo al momento del training.

Tutti i risultati sono mediati su 3 seed $\{42, 123, 456\}$. Gli array grezzi sono salvati in `results/` come `.npz` e **tutte le tabelle di questo report sono generate da quei file** (`report/make_report_tables.py`): nessun numero è trascritto a mano.

3.5 Origine del codice

I **quattro algoritmi tabellari** sono adattati dalle esercitazioni del corso (`control.py`, `prediction.py`): la regola di aggiornamento è la medesima, riadattata all'API Gymnasium moderna e agli stati discretizzati. Il **DQN** (replay buffer, loop di training, target network, Double DQN), il **discretizzatore**, l'infrastruttura multi-seed e tutti gli script sono **scritti da zero**: il corso forniva solo un DQN basato su Stable-Baselines3, che la consegna vieta per il core. La Nature-CNN replica l'architettura di Mnih et al. (2015) con implementazione PyTorch originale. Nessuna riga di Stable-Baselines3 è presente nel progetto.

4 Setup sperimentale

4.1 Hardware, librerie, iperparametri

Gli esperimenti tabellari sono eseguiti su CPU; il DQN su GPU NVIDIA RTX 5060 (8 GB di VRAM, su una macchina con 32 GB di RAM), che richiede la build PyTorch CUDA 12.8. Le librerie utilizzate sono Gymnasium (con ALE), PyTorch, NumPy e Matplotlib; le dipendenze sono gestite con `uv` e bloccate in `uv.lock`. Nessuna libreria RL di alto livello è impiegata nel core, come richiesto dalla consegna.

Tabella 2: Iperparametri principali. I default tabellari seguono Sutton & Barto; quelli del DQN sono ridotti rispetto al paper Nature per rientrare nel budget di calcolo.

	Tabellari (CartPole / Acrobot)	DQN (Pong)
Budget	10 000 / 20 000 episodi	200 000 step ambiente
α	0.1	10^{-4} (Adam)
γ	0.99	0.99
ε	0.1 (costante)	$1.0 \rightarrow 0.02$ (lineare, 100k step)
Replay buffer	—	100 000 transizioni
Batch / target update	—	32 / ogni 1000 step
Learning starts	—	10 000 step
Seed	$\{42, 123, 456\}$ — 3 run per ogni esperimento	

Acrobot riceve il doppio degli episodi rispetto a CartPole perché il suo spazio degli stati è circa cinque volte più grande e il reward è sparso: con 10 000 episodi le curve non erano ancora entrate in regime.

Esperimenti. *CartPole*: Q-Learning vs SARSA; sensibilità a ε, α, γ per *entrambi* gli algoritmi; MC vs TD(0) su policy appresa e casuale; tre casi di fallimento; ε costante vs annealing. *Acrobot*: gli stessi confronti, più l'effetto della risoluzione di discretizzazione (4/6/8/10 bin). *Pong*: ablazione a cinque configurazioni (DQN completo, senza target network, senza replay, senza entrambi, Double DQN) più la baseline casuale.

4.2 Come vanno lette le figure

Le curve riportano il return per episodio, mediato sui 3 seed e lisciato con media mobile. Nei confronti a due curve (controllo, annealing, predizione, ablazione Pong) la banda ombreggiata rappresenta la deviazione standard $\pm 1\sigma$ tra i seed.

Nelle figure di **sensibilità**, dove compaiono quattro curve sovrapposte, le bande sono state *deliberatamente omesse*: le regioni ombreggiate si sovrapporrebbero rendendo il grafico illeggibile. Ogni curva resta comunque la media sugli stessi 3 seed, e la dispersione è riportata numericamente nelle tabelle.

Le tabelle riportano la performance "finale" come media sull'**ultimo 10%** degli episodi di ciascun seed, aggregata poi tra seed: si tratta di una stima dell'asintoto più robusta rispetto al valore degli ultimissimi episodi, che sono intrinsecamente rumorosi. Nelle curve di Pong, dove seed diversi producono un numero diverso di episodi, la curva viene tracciata solo fino al seed più corto, in modo che ogni punto rappresenti una media su tutti e 3 i seed.

5 Risultati

Tutti i valori numerici provengono dagli array `.npz` salvati dai runner e sono calcolati da `make_report_tables.py`.

5.1 CartPole — controllo: Q-Learning vs SARSA

Entrambi gli algoritmi mostrano un apprendimento netto (Fig. 1, Tab. 3), passando da un return iniziale di $\approx$ 27–41 a 223.6 $\pm$ 49.4 per Q-Learning e 258.3 $\pm$ 31.1 per SARSA.

Le curve sono tuttavia ancora in salita al termine delle 10 000 epoche di training, senza aver raggiunto un plateau: considerando che il massimo dell'ambiente è 500, il vincolo è chiaramente il budget computazionale e non l'algoritmo, poiché la griglia a 10^4 stati richiede un'esplorazione quasi esaustiva.

Il fatto che SARSA superi Q-Learning e presenti una dispersione tra seed inferiore ($\pm$31 contro $\pm$49) è l'opposto di ciò che ci si aspetterebbe, dato che Q-Learning apprende la policy *ottima* mentre SARSA opera senza decay di ε. La spiegazione risiede nel fatto che la curva misura la performance *durante* il training, con $\varepsilon = 0.1$ attivo: Q-Learning valuta sempre la policy greedy ignorando che il 10% delle azioni sarà casuale, con il rischio di errori conseguenti, mentre SARSA include quel rischio nel target e apprende una policy che tollera le proprie deviazioni esplorative. Si riconosce la dinamica del CliffWalking (S&B Fig. 6.5), attenuata ma chiaramente visibile. Naturalmente, a training concluso e con ε azzerato, Q-Learning risulta superiore, poiché la sua policy greedy è ottimale.

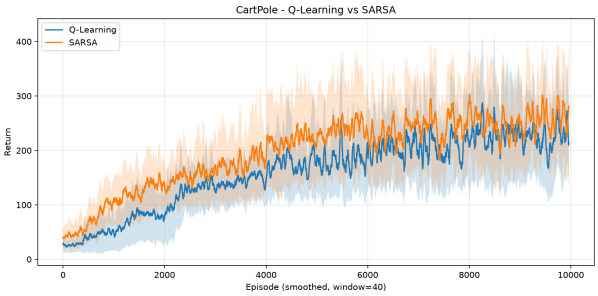


Figura 1: CartPole: Q-Learning vs SARSA a parità di iperparametri (media su 3 seed, banda $\pm 1\sigma$). SARSA (arancio) domina per l'intera durata del training; entrambe le curve sono ancora in salita a 10 000 episodi.

Tabella 3: CartPole, controllo: Q-Learning vs SARSA a parità di iperparametri ($\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon = 0.1$), media su 3 seed. Il return massimo dell'ambiente è 500; entrambe le curve sono ancora in salita a fine budget.

Algoritmo	Return iniziale (primi 100 ep)	Return finale (ultimo 10% ep, media $\pm$ std)	Episodi/seed
Q-Learning	26.6	223.6 $\pm$ 49.4	10000
SARSA	40.8	258.3 $\pm$ 31.1	10000

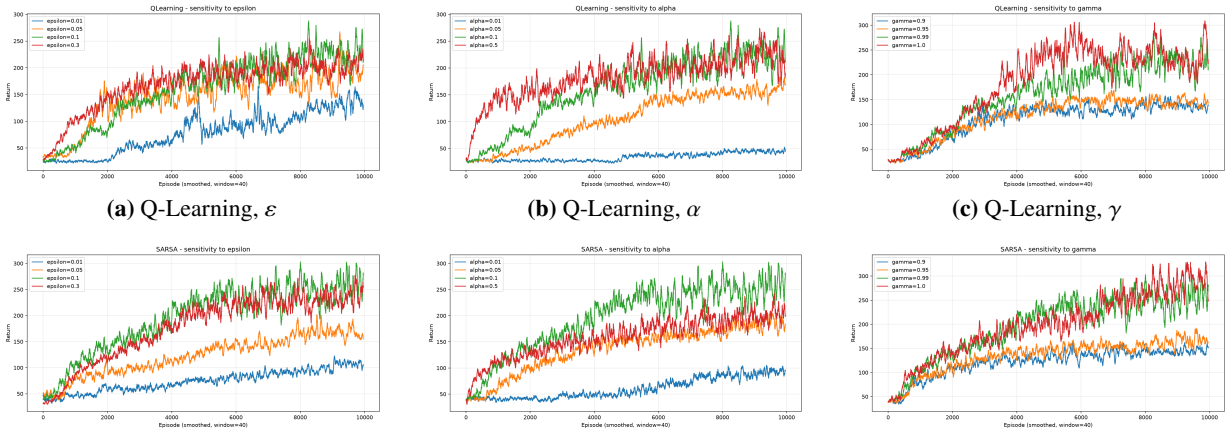
5.2 CartPole — sensibilità a ε, α, γ

La Fig. 2 riporta le sei sweep di sensibilità.

- **γ produce l'effetto più netto.** Valori alti dominano per entrambi gli algoritmi: $\gamma \in \{0.99, 1.0\}$ raggiunge 230–280, mentre $\gamma \in \{0.9, 0.95\}$ si arresta a 140–165. Il risultato è coerente con la natura di CartPole, che è un compito di *sopravvivenza* con reward +1/passo: il valore di uno stato coincide con la durata della sopravvivenza, e con $\gamma = 0.9$ l'orizzonte effettivo $\approx 1/(1-\gamma) = 10$ passi è troppo corto perché l'agente possa anticipare la caduta.
- **ε è necessario, ma valori elevati non danneggiano.** $\varepsilon = 0.01$ risulta il peggiore per entrambi gli algoritmi (105–130), mentre $\varepsilon = 0.3$ *non* degrada la performance (220–250, comparabile a 0.1): la

presenza di un reward denso garantisce un margine di tolleranza verso l'esplorazione, a differenza di quanto si osserverà su Acrobot (Sez. 5.7). L'effetto di ε dipende dunque dalla struttura del reward e non è una proprietà intrinseca dell'algoritmo.

- α **è il parametro su cui i due algoritmi divergono.** Per Q-Learning $\alpha = 0.5$ e $\alpha = 0.1$ convergono allo stesso livello (200–220), con 0.5 più veloce nelle fasi iniziali. Per *SARSA* invece $\alpha = 0.5$ parte con valori elevati ma **satura al di sotto** di $\alpha = 0.1$ (≈ 210 contro ≈ 260), poiché SARSA, essendo on-policy, presenta nel target $q(S_{t+1}, A_{t+1})$ la casualità dell'azione A_{t+1}: un learning rate elevato insegue quel rumore anziché mediarlo. Il valore $\alpha = 0.01$ risulta invece troppo lento per entrambi (45–95).



(a) Q-Learning, ε **(b)** Q-Learning, α **(c)** Q-Learning, γ

(d) SARSA, ε **(e)** SARSA, α **(f)** SARSA, γ

Figura 2: CartPole: sweep di sensibilità (media su 3 seed; bande omesse per leggibilità, Sez. 4.2). Riga superiore Q-Learning, inferiore SARSA. Un γ elevato domina in entrambi i casi; un ε troppo basso penalizza, mentre $\varepsilon = 0.3$ non danneggia. La differenza più rilevante tra i due algoritmi emerge nel pannello α: per SARSA (e) $\alpha = 0.5$ (rosso) parte veloce ma satura *sotto* $\alpha = 0.1$ (verde), mentre per Q-Learning (b) i due valori si equivalgono.

5.3 CartPole — ε costante vs annealing

L'ipotesi di partenza era che l'annealing fosse di scarsa utilità su CartPole, dato che l'ambiente presenta un reward denso e non è limitato dall'esplorazione. Il beneficio si è rivelato presente, ma concentrato soprattutto sulla *stabilità*: l'annealing ($1.0 \to 0.01$) chiude a 243.8 ± 23.9 contro 223.6 ± 49.4 del costante (Fig. 3), considerando l'ultimo 10% degli episodi. Il guadagno in media è modesto (+20), ma la dispersione tra seed si *dimezza*: ridurre ε nelle fasi finali del training elimina il rumore che la policy si porta dietro, al prezzo di un avvio più lento che si protrae fino a circa 3 500 episodi.

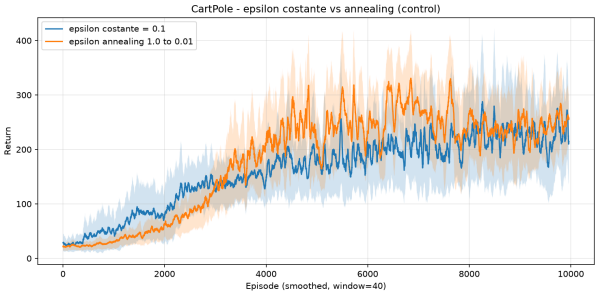


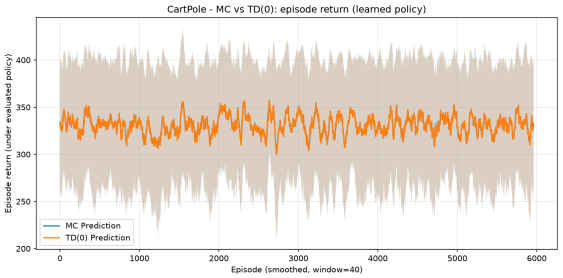
Figura 3: CartPole: ε costante = 0.1 vs annealing $1.0 \to 0.01$. L'annealing paga un avvio più lento ma poi sorpassa il costante e chiude con banda $\pm 1\sigma$ visibilmente più stretta.

5.4 CartPole — MC vs TD(0): bias, varianza, stabilità

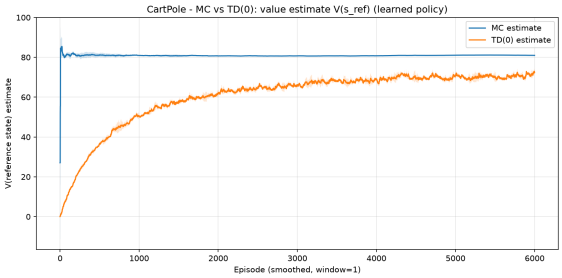
Il return non è in grado di distinguere i due stimatori (Fig. 4a): le curve risultano *identiche per costruzione*, poiché MC e TD(0) valutano la *stessa* policy greedy fissa e generano quindi i medesimi episodi (return ≈ 330). La curva di MC è letteralmente nascosta sotto quella di TD, a conferma che la differenza tra i due metodi non risiede nel return ma nelle stime $v(s)$. Per inciso, quei ≈ 330 passi sono ottenuti dalla policy greedy *senza* esplorazione, rispetto ai ≈ 224 della curva di training: gran parte del divario dal massimo è il costo di ε, non un'incapacità della policy.

La differenza emerge chiaramente nelle stime di $V(s_{\text{ref}})$ (Fig. 4b), che riproducono il quadro classico di S&B §6.2. **MC** raggiunge il valore corretto (81.07 ± 0.17) in poche decine di episodi ma con un picco iniziale violento: la stima è *non distorta* e ad *alta varianza*, poiché ogni aggiornamento impiega il return completo di una traiettoria di ≈ 330 passi. **TD(0)** parte da 0 e sale lentamente con estrema regolarità, arrestandosi a 70.50 ± 0.28 dopo 6 000 episodi, ancora il 13% al di sotto di MC: la stima è *distorta* (a causa del bootstrap su una stima ancora inaccurata) ma a *bassa varianza*.

Il bias di TD(0) non è tuttavia una costante, bensì dipende dalla policy valutata, ed è questo il risultato che giustifica l'aver valutato anche la policy casuale (Fig. 5). Con episodi di soli ≈ 22 passi, lo stato di riferimento è molto vicino alla terminazione e TD(0) *raggiunge* MC entro $\approx 5\,000$ episodi (17.04 contro 17.43), facendo essenzialmente scomparire il bias. Con la policy appresa, invece, gli episodi sono quindici volte più lunghi e il bootstrap deve propagare il valore un passo alla volta: la distorsione di TD(0) risulta quindi una funzione della *distanza temporale* tra lo stato e il reward, e non un difetto intrinseco dell'algoritmo.



(a) Return: identico per costruzione.



(b) Stima $V(s_{\text{ref}})$: qui i due divergono.

Figura 4: CartPole, MC vs TD(0) sulla **policy appresa**. (a) Poiché la policy è la stessa, gli episodi sono identici e il return non può fornire informazioni sugli stimatori. (b) Le stime rivelano invece il compromesso bias/varianza: MC (blu) è non distorto ma con alta varianza iniziale, TD(0) (arancio) è regolare ma distorto verso l'inizializzazione e non ancora convergente dopo 6 000 episodi.

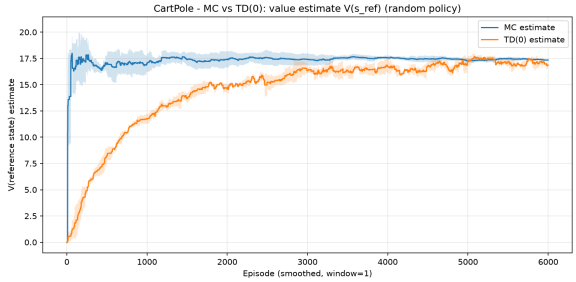


Figura 5: CartPole, MC vs TD(0) sulla **policy casuale**. Con episodi corti (≈ 22 passi) il bootstrap propaga rapidamente e TD(0) *raggiunge* MC: il bias dipende dalla distanza tra lo stato e la terminazione. Da confrontare con la Fig. 4b.

Le figure del return sulla policy casuale e su Acrobot esistono in `figures/` ma non sono riportate: mostrano lo stesso fenomeno di Fig. 4a (curve sovrapposte per costruzione) e non aggiungono informazione.

5.5 Acrobot — controllo

Entrambi gli algoritmi migliorano da ≈ -470 (prossimo al livello casuale di -500) fino a -341.4 ± 27.8 per Q-Learning e -348.6 ± 3.2 per SARSA, restando tuttavia molto lontani dall'ottimo di ≈ -100: l'agente apprende qualche spinta utile, ma non uno swing-up efficiente (Fig. 6, Tab. 4).

Il dato più istruttivo riguarda la **forma** della curva: il grosso del miglioramento avviene entro i primi $\approx 5\,000$ episodi, dopodiché le curve oscillano attorno a $-340/-350$ *senza ulteriori progressi* per altri 15 000 episodi. Non si tratta di lentezza ma di un tetto strutturale: raddoppiare il budget non lo sposterebbe. Con 46 656 stati e un reward informativo solo quando l'obiettivo viene raggiunto per caso, la Q-table a 6 bin non possiede la risoluzione necessaria a rappresentare una politica migliore, come la sezione successiva verifica.

Merita attenzione l'inversione rispetto a CartPole sulla varianza: qui è **SARSA** a risultare drasticamente più stabile (± 3.2 contro ± 27.8). La stabilità relativa dei due algoritmi non è dunque una loro proprietà intrinseca ma dipende dall'ambiente; la Sez. 7 riprende il punto.

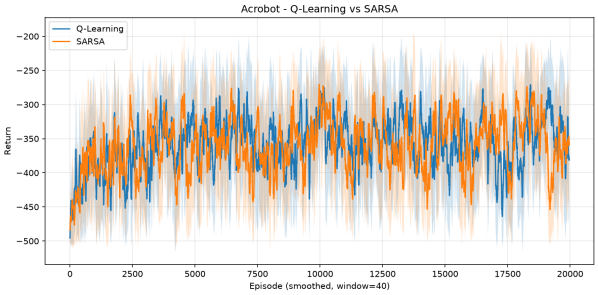


Figura 6: Acrobot: Q-Learning vs SARSA. Si osserva un miglioramento reale rispetto al livello casuale (-500), ma con un plateau precoce ($\approx 5\,000$ episodi) molto lontano dall'ottimo (≈ -100).

Tabella 4: Acrobot, controllo: Q-Learning vs SARSA, media su 3 seed. Policy casuale ≈ -500; un buon swing-up raggiunge ≈ -100.

Algoritmo	Return iniziale (primi 100 ep)	Return finale (ultimo 10% ep, media ± std)	Episodi/seed
Q-Learning	-468.0	-341.4 ± 27.8	20000
SARSA	-473.2	-348.6 ± 3.2	20000

5.6 Acrobot — risoluzione della discretizzazione

Questo esperimento ha smentito l'ipotesi di partenza ed è il risultato più istruttivo del progetto. L'ipotesi era che aumentare il numero di bin *peggiorasse* l'apprendimento, poiché lo spazio degli stati cresce come bin^6 e a 10 bin (10^6 stati) la tabella sarebbe diventata irrimediabilmente sparsa, secondo la classica *curse of dimensionality*.

I dati mostrano l'**opposto** in modo monotono e non ambiguo (Fig. 7): dopo 20 000 episodi, **10 bin risulta la configurazione migliore** (≈ -280) e **4 bin la peggiore** (≈ -430), con 8 e 6 bin ordinatamente nel mezzo.

La spiegazione risiede nel costo dell'*aliasing*: le dinamiche di Acrobot richiedono precisione angolare, e con 4 bin per dimensione configurazioni del pendolo che richiedono coppie *opposte* finiscono nello *stesso* stato discreto. La Q-table non può rappresentare politiche diverse per stati che non sa distinguere, e nessun numero di episodi può rimediare a questo limite, come confermato dalla curva a 4 bin che resta piatta e rumorosa per l'intera durata dell'esperimento. Questo costo di aliasing domina quello della sparsità, e costituisce anche la giustificazione empirica del passaggio al DQN, poiché per ottenere insieme precisione e generalizzazione è necessario un approssimatore di funzione.

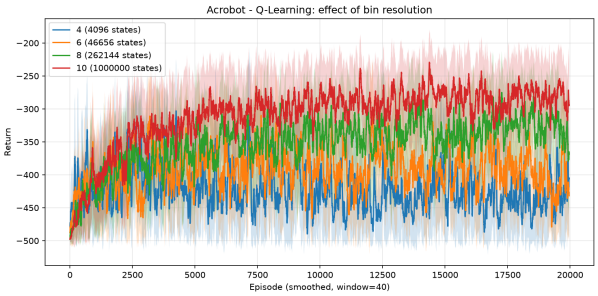


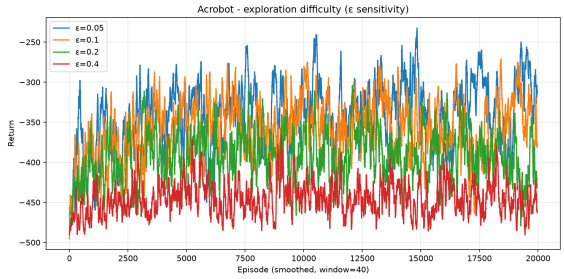
Figura 7: Acrobot: effetto della risoluzione di discretizzazione (Q-Learning, 20 000 episodi). Contrariamente all'ipotesi, 10 bin (10^6 stati, rosso) è la configurazione *migliore* e 4 bin (blu) la peggiore, con ordinamento monotono: la griglia grossolana non riesce a distinguere stati che richiedono coppie opposte.

5.7 Acrobot — esplorazione: ε costante e annealing

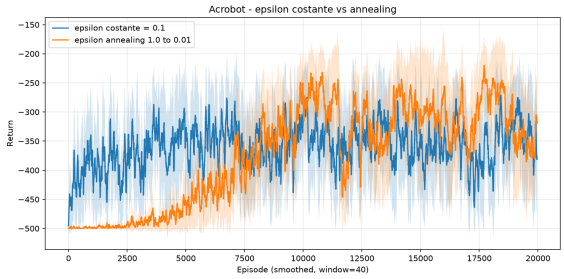
L'attesa era che un reward sparso richiedesse più esplorazione, ma la sweep a ε costante mostra il contrario (Fig. 8a): i valori *bassi* prevalgono ($\varepsilon = 0.05$ raggiunge ≈ -310) e $\varepsilon = 0.4$ risulta nettamente il peggiore (≈ -450), nuovamente in ordine monotono.

Il problema non è che l'esplorazione sia inutile, bensì che **l'esplorazione *costante* è lo strumento sbagliato**: con $\varepsilon = 0.4$ l'agente continua a iniettare coppie casuali *anche dopo* aver trovato uno swing-up efficace, distruggendolo sistematicamente ad ogni episodio.

L'annealing conferma questa lettura e chiude il cerchio (Fig. 8b): lo schedule $1.0 \to 0.01$ raggiunge -316.3 ± 24.5 contro -341.4 ± 27.8 del miglior ε costante, rappresentando il guadagno più significativo ottenuto su Acrobot senza modificare l'algoritmo. Il prezzo è visibile: per i primi $\approx 5\,000$ episodi la curva con annealing è inchiodata a -500 (con $\varepsilon \approx 1$ l'agente si comporta in modo casuale), per poi sorpassare il costante attorno ai 9 000 episodi e restare stabilmente al di sopra. Il confronto con CartPole, dove lo stesso schedule produce un guadagno modesto, isola il meccanismo: l'annealing risulta vantaggioso là dove l'esplorazione rappresenta davvero il collo di bottiglia.



(a) ε costante: valori bassi prevalgono.



(b) Annealing $1.0 \to 0.01$ vs costante 0.1.

Figura 8: Acrobot, esplorazione. (a) A ε *costante* i valori bassi prevalgono: un ε elevato tenuto fisso continua a distruggere lo swing-up già appreso. (b) L'esplorazione *decrescente* si rivela la scelta giusta: investe $\approx 5\,000$ episodi a -500 ma poi supera stabilmente il miglior ε costante. Complessivamente, i due pannelli indicano che il problema non risiede nella *quantità* di esplorazione, ma nel *momento* in cui viene impiegata.

5.8 Acrobot — MC vs TD(0)

Su Acrobot il bias di TD(0) è ancora più marcato che su CartPole (Fig. 9), confermando l'interpretazione precedente. MC converge quasi immediatamente a -91.17 ± 0.27, mentre TD(0) parte da 0, *scende lentamente* e dopo 6 000 episodi si trova ancora a -62.00 ± 0.03. Lo stato di riferimento dista ≈ 90 passi dall'obiettivo e TD propaga il valore un passo per volta: per uno stato così distante dal reward il bootstrap risulta estremamente lento e la stima resta ancorata all'inizializzazione.

La banda di TD(0) è così stretta da risultare invisibile nel grafico (±0.03), mentre quella di MC è ampia nei primi episodi. Si tratta del compromesso bias/varianza nella sua forma più pura: TD(0) è molto *preciso* nel fornire una risposta *sbagliata*, e necessita di molti episodi per convergere a quella corretta.

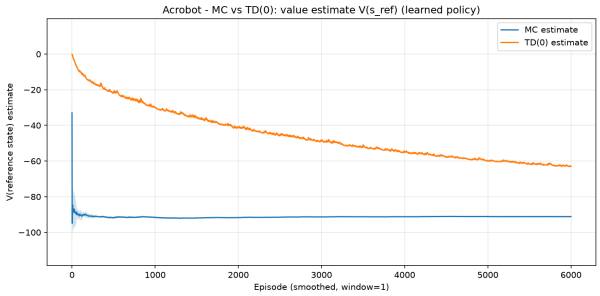


Figura 9: Acrobot, MC vs TD(0): stima $V(s_{\text{ref}})$ della policy appresa. MC (blu) converge rapidamente a ≈ -91, mentre TD(0) (arancio) scende da 0 con estrema regolarità ma dopo 6 000 episodi si trova ancora a ≈ -62.

5.9 Pong — ablazione del DQN

I risultati sono riassunti in Fig. 10 e Tab. 5, con la baseline casuale a ≈ -20.4 e un budget di soli 200k step. Oltre alla media su 3 seed, riporto tra parentesi *quanti seed su 3* hanno effettivamente mostrato apprendimento (return finale superiore a -19): su soli 3 seed la deviazione standard è essa stessa molto rumorosa e può produrre barre d'errore ingannevolmente strette, mentre il conteggio dei seed che apprendono permette di distinguere "la configurazione non impara" da "impara solo a volte", una differenza che la sola media tenderebbe ad appiattire.

- **DQN completo**: -13.5 ± 1.1 (3/3). L'apprendimento è netto e interessa *tutti* i seed, soddisfacendo con ampio margine il requisito della consegna di mostrare apprendimento rispetto alla policy casuale.
- **Double DQN** (bonus): -17.6 ± 2.3 (2/3). L'apprendimento è presente ma più lento rispetto al DQN completo, con un seed che non decolla.
- **Senza target network**: -20.0 ± 1.4 (1/3). Si tratta del risultato più significativo: due seed su tre *restano al livello casuale* e uno solo apprende.
- **Senza replay / senza entrambi**: -20.9 ± 0.0 e -20.9 ± 0.0 (0/3). *Nessun apprendimento*: la curva resta piatta al livello casuale su ogni seed.

Tabella 5: Ablazione DQN su Pong: performance finale dopo 200k step ambiente, media ± std su 3 seed. Policy casuale ≈ -20.4.

Configurazione	Return finale (ultimo 10% ep, media ± std)	Episodi/seed (media)
DQN completo (replay + target)	-13.5 ± 1.1	137
Double DQN (bonus)	-17.6 ± 2.3	174
Senza target network	-20.0 ± 1.4	206
Senza experience replay	-20.9 ± 0.0	241
Senza target + senza replay	-20.9 ± 0.0	239

Entrambi i componenti si rivelano quasi indispensabili (Fig. 10). Senza il *replay*, la rete si addestra sul flusso di transizioni consecutive fortemente correlate e in 200k step non apprende nulla su nessun seed, producendo curve piatte. La *target network*, d'altra parte, non funge da semplice acceleratore: *senza di essa l'apprendimento diventa una lotteria*. Il valore medio (-20.0) nasconde il fenomeno reale, leggibile nella banda enorme e nell'analisi per-seed: un solo seed sale a ~ -18 mentre gli altri due restano piatti a ~ -21. La causa risiede nel "bersaglio mobile" portato all'estremo dal budget ridotto: ogni passo di gradiente sposta anche il target, ponendo l'addestramento in un equilibrio instabile il cui esito è determinato dalle

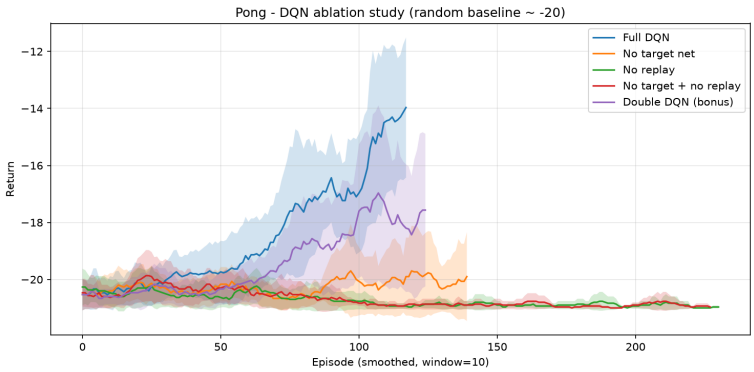


Figura 10: Pong: ablazione DQN (200k step, 3 seed, baseline casuale ≈ -20). La figura contiene entrambe le ablazioni richieste dalla consegna: *con vs senza replay* (blu vs verde) e *con vs senza target network* (blu vs arancio). Solo il DQN completo (blu) apprende su tutti i seed; il Double DQN (viola) apprende più lentamente; senza target network (arancio) la banda di dimensioni elevate segnala che la maggioranza dei seed non decolla; le due configurazioni senza replay (verde e rosso) restano piatte. Le figure per-ablazione isolate sono disponibili in `figures/pong/`.

condizioni iniziali controllate dal seed (pesi della rete e prime esperienze esplorative). La lettura corretta non è dunque "replay necessario, target utile", bensì: *il replay rende l'apprendimento possibile, la target network lo rende affidabile*; solo con entrambi il DQN apprende su tutti e 3 i seed. A conferma, un segnale indiretto: il DQN completo gioca *meno* episodi (137 contro ≈ 240), poiché apprendere allunga gli scambi e a parità di step le partite risultano meno numerose.

Il **Double DQN** non supera il DQN base, anzi ne peggiora leggermente la performance (-17.6 contro -13.5, con un seed che non converge). L'algoritmo corregge la sovrastima dovuta al max su stime rumorose, fenomeno che si accumula su orizzonti lunghi; tuttavia, a 200k step (lo 0.4% del budget del paper Nature) quel danno non ha ancora avuto modo di manifestarsi, mentre il disaccoppiamento tra selezione e valutazione introduce un ulteriore elemento di rumore. Il bonus è implementato e funzionante, ma in questo regime di training abbreviato non sussiste il problema che è progettato per risolvere.

6 Failure analysis

La consegna chiede almeno un caso di fallimento: ne vengono riportati **tre**, ottenuti portando all'estremo un iperparametro alla volta su CartPole con Q-Learning (Fig. 11). Ciascuno è scelto perché rompe un meccanismo *diverso*: il passo di aggiornamento, l'esplorazione e l'assegnazione del credito. In tutti e tre i casi il codice gira senza errori, ed è esattamente questo il punto: la correttezza dell'implementazione non garantisce l'apprendimento.

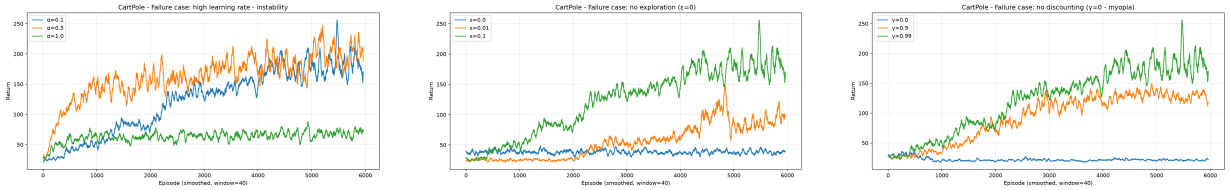
1. $\alpha = 1.0$ — instabilità (mancata convergenza). L'aggiornamento *sostituisce* completamente la stima ad ogni passo ($q(S_t, A_t) \leftarrow R_{t+1} + \gamma \max_{a'} q(S_{t+1}, a')$): la media incrementale scompare e Q non può mediare sulla stocasticità delle transizioni, finendo per inseguire l'ultimo campione osservato. L'agente si blocca a ≈ 65 per l'intera durata dei 6 000 episodi.

La lettura più importante riguarda però un aspetto diverso: il fallimento è specifico del valore *estremo*, non dell'"α alto" in generale. Nella stessa figura $\alpha = 0.5$ non solo non fallisce, ma risulta la curva *migliore* (≈ 210, superiore ad $\alpha = 0.1$). Se avessi testato soltanto $\{0.1, 1.0\}$ avrei concluso, erroneamente, che "α grande destabilizza": la transizione non è graduale e la soglia di instabilità si colloca tra 0.5 e 1.0.

2. $\varepsilon = 0$ — comportamento subottimale. L'agente opera in modo greedy fin dal primo istante su una Q-table inizializzata a zero: si fissa sulla prima azione che sembra funzionare senza mai provare le alternative. La curva resta piatta a ≈ 40, mentre con $\varepsilon = 0.1$ si raggiunge ≈ 170. Degno di nota il caso intermedio $\varepsilon = 0.01$, che non fallisce del tutto ma resta bloccato per $\approx 2\,500$ episodi prima di sbloccarsi e arrivare a ≈ 100. Un'esplorazione insufficiente non impedisce l'apprendimento in senso assoluto, ma lo rende estremamente inefficiente (S&B §2.1).

3. $\gamma = 0$ — miopia (fallimento del credit assignment). Si tratta del caso più interessante, poiché il fallimento non è dovuto a lentezza ma è *strutturalmente impossibile* da superare. Con $\gamma = 0$ il target si riduce al reward immediato; tuttavia, su CartPole il reward immediato è +1 ad *ogni* passo non terminale, *qualunque* azione venga scelta. Tutte le coppie stato-azione collassano pertanto sullo stesso valore, la policy greedy non ha modo di preferire un'azione all'altra e degenera in una policy casuale: la curva resta piatta a ≈ 22, vale a dire *al di sotto* del punto di partenza delle altre configurazioni.

Questo caso mostra concretamente *a cosa serve* il discount factor: non semplicemente a "pesare meno il futuro", ma a creare il canale attraverso cui l'informazione sulla caduta futura risale fino alle azioni che l'hanno causata. Senza $\gamma > 0$ quel canale non esiste, e nessun numero di episodi può rimediare.



(a) $\alpha = 1.0$: instabilità. **(b)** $\varepsilon = 0$: subottimale. **(c)** $\gamma = 0$: miopia.

Figura 11: CartPole, i tre casi di fallimento (Q-Learning, media su 3 seed). (a) $\alpha = 1.0$ (verde) si blocca a ≈ 65, mentre $\alpha = 0.5$ (arancio) è la curva migliore: la soglia di instabilità si colloca tra i due valori, non genericamente "in alto". (b) $\varepsilon = 0$ (blu) resta piatto a ≈ 40, mentre $\varepsilon = 0.01$ (arancio) apprende con $\approx 2\,500$ episodi di ritardo. (c) $\gamma = 0$ (blu) è piatto a ≈ 22: con reward +1 ad ogni passo tutte le azioni risultano equivalenti e la policy greedy è indistinguibile da una casuale.

Un fallimento non provocato: il tetto di Acrobot. Ai tre fallimenti indotti va aggiunto quello *spontaneo* della Sez. 5.5: su Acrobot le curve si arrestano a ≈ -345 senza ulteriori miglioramenti per 15 000 episodi. Non si tratta di un bug né di un iperparametro mal configurato, bensì del limite rappresentativo della Q-table

discretizzata; la Sez. 5.6 lo dimostra facendo variare l'unica variabile in grado di spostarlo: il numero di bin.

7 Analisi critica

Questa sezione raccoglie le letture *trasversali*, che nessun singolo esperimento mostra da solo.

7.1 SARSA vs Q-Learning: la teoria non predice il vincitore

Il confronto ha prodotto esiti *opposti* nei due ambienti, ed è istruttivo proprio per questo. Su **CartPole** SARSA prevale su entrambi i fronti (return 258 vs 224, deviazione standard ±31 vs ±49), mentre su **Acrobot** Q-Learning ottiene un return leggermente migliore (−341 vs −349) ma SARSA risulta quasi dieci volte più stabile tra i seed (±3.2 vs ±27.8).

La differenza teorica è sempre la medesima: il target di Q-Learning impiega il max e ignora l'esplorazione, quello di SARSA impiega l'azione realmente scelta e la include. Le *conseguenze*, tuttavia, dipendono dall'ambiente. Là dove l'esplorazione è rischiosa (CartPole: una mossa casuale può far cadere l'asta), SARSA guadagna perché apprende una policy robusta al proprio rumore esplorativo. Là dove il reward è sparso (Acrobot), il max di Q-Learning amplifica le rare traiettorie fortunate che raggiungono l'obiettivo: quali siano dipende dal seed, il che spiega i picchi più elevati ma anche la varianza molto superiore. Concludere che "off-policy è meglio" non ha pertanto valore predittivo: ciò che conta è come il rischio dell'esplorazione interagisce con la struttura del reward.

7.2 Perché le curve non convergono mai davvero

Nessuna curva tabellare si appiattisce del tutto, e non deve farlo: con ε e α *costanti* l'agente continua a esplorare e pesa le esperienze recenti in modo esponenziale (media a pesi mobili, non media esatta), e dunque la fluttuazione asintotica è un comportamento atteso (S&B §2.5). Il fatto che l'annealing di ε riduca sensibilmente la varianza tra seed su entrambi gli ambienti ne costituisce la conferma sperimentale.

7.3 Due ipotesi smentite, e perché sono state mantenute

I risultati sui bin e su ε in Acrobot contraddicono entrambi la previsione iniziale. Non ho ritoccato né gli esperimenti né le ipotesi: i risultati sono riproducibili sui 3 seed, *monotoni* nel parametro (il che esclude che siano rumore) e ammettono una spiegazione meccanicistica coerente.

Il punto interessante è che le due "regole" smentite sono vere *in astratto*: la curse of dimensionality esiste, e un reward sparso richiede effettivamente più esplorazione. Si tratta però di affermazioni *asintotiche*, mentre un esperimento si svolge a budget finito e su un ambiente specifico: esattamente il tipo di errore che solo la verifica sperimentale è in grado di intercettare.

7.4 Limiti metodologici riconosciuti

- **Predizione: misura locale.** Il confronto MC vs TD(0) traccia $V(s_{\text{ref}})$ di *un* singolo stato; una misura globale (RMSE di V su tutti gli stati, come nel *random walk* del corso) richiederebbe un V di riferimento che in questi ambienti non è disponibile. L'attenuante è che valutare *due* policy diverse ha comunque permesso di isolare *da cosa* dipende il bias di TD(0).
- **Asse x in "episodi" nell'ablazione Pong.** A parità di 200k step le diverse configurazioni producono numeri diversi di episodi, per cui confrontarle allo stesso indice non costituisce un confronto a perfetta parità di calcolo. Un asse in step ambiente sarebbe più rigoroso; la conclusione qualitativa non cambia, dal momento che le curve senza replay restano piatte per l'intera durata.
- **Persistenza parziale dei dati grezzi.** Controllo, annealing, predizione e ablazione Pong sono salvati in `.npz`; le sweep di sensibilità e i fallimenti producono solo figure, e pertanto le tabelle utilizzano unicamente dati persistiti.
- **Bande assenti nelle figure di sensibilità**, per ragioni di leggibilità (Sez. 4.2).

- **Il DQN è valutato durante il training**, con $\varepsilon > 0$: si tratta di una stima leggermente *pessimistica* della policy greedy. I checkpoint sono salvati, e una valutazione greedy separata sarebbe fattibile senza necessità di riaddestrare.

7.5 Problemi implementativi incontrati

Distinti dai limiti intrinseci del RL, questi sono ostacoli concreti che è stato necessario risolvere:

- **GPU Blackwell (RTX 5060)**: richiede la build PyTorch CUDA 12.8; con build precedenti la GPU non viene rilevata e il training ricade silenziosamente su CPU.
- **Pipeline Atari, tre insidie**: (i) l'ambiente `-v5` possiede già un frame-skip interno che, combinato con `AtariPreprocessing`, produrrebbe uno skip effettivo di 16 frame, rendendo necessario forzare `frameskip=1`; (ii) in Gymnasium 1.x `FrameStack` si chiama `FrameStackObservation`; (iii) i frame impilati sono già in formato channel-first `(4,84,84)` e non devono essere trasposti. Un errore su uno qualsiasi dei tre produce un crash oppure, peggio ancora, input silenziosamente malformati.
- **Memoria del replay buffer**: 1M di transizioni Atari richiederebbero ≈ 28 GB, pertanto il buffer è stato limitato a 100k transizioni mantenendo i frame in formato `uint8`.
- **DQN non riproducibile per un RNG nascosto.** Run che avrebbero dovuto essere identiche divergevano: l'azione esplorativa viene estratta da `env.action_space.sample()`, e lo spazio delle azioni dispone di un *RNG separato* che né `set_seed` né `reset(seed=)` inizializzano. Nelle fasi iniziali ($\varepsilon \approx 1$) quasi tutta l'esperienza proveniva da quel RNG non seminato. Aggiunto `env.action_space.seed(seed)`; i risultati Pong riportati provengono da questo run corretto e riproducibile.
- **Episodio troncato contato come completo.** Quando il budget di step si esauriva a metà partita, il return parziale (troppo ottimistico) veniva registrato come episodio completo, producendo un picco spurio verso l'alto sul bordo destro di ogni curva. Corretto scartando l'episodio non terminato.
- **Stato di riferimento impossibile** (Sez. 3.3): il grafico piatto a zero era formalmente corretto; era la domanda a essere mal posta.

Nel complesso i risultati mostrano apprendimento, confronti corretti e fallimenti analizzati, e in questo sono adeguati agli obiettivi della consegna. Non si tratta tuttavia di risultati "research-grade", e i relativi divari sono dichiarati apertamente nelle conclusioni.

8 Conclusioni

Si risponde esplicitamente alle tre domande richieste dalla consegna.

8.1 Cosa non ha funzionato

- **Acrobot non decolla**: il plateau a ≈ -345, contro un ottimo di ≈ -100, persiste senza miglioramenti per 15 000 episodi. Raddoppiare il budget non produce effetti, poiché il limite risiede nella risoluzione della Q-table (Sez. 5.5).
- **CartPole non raggiunge 500**: con 10^4 stati il tetto in 10 000 episodi si colloca a circa metà del massimo. Anche in questo caso il vincolo è di natura rappresentativa, non algoritmica.
- **I tre fallimenti indotti** (Sez. 6): $\alpha = 1.0$ non converge, $\varepsilon = 0$ resta subottimale e $\gamma = 0$ rende l'apprendimento strutturalmente impossibile, pur in assenza di errori nel codice.
- **Double DQN non supera il DQN base** a 200k step, producendo anzi un lieve peggioramento: il maximization bias che dovrebbe correggere non ha ancora avuto il tempo di accumularsi (Sez. 5.9).
- **Il DQN non era inizialmente riproducibile**: lo spazio delle azioni dispone di un RNG separato che né `set_seed` né `reset(seed=)` toccano, e da cui proviene ogni azione esplorativa; due run con lo stesso seed divergevano. Scoperto e corretto (Sez. 7).
- **Il primo confronto MC vs TD(0) non misurava nulla**: graficava il return della policy condivisa, identico per costruzione tra i due stimatori: formalmente corretto ma privo di qualsiasi contenuto informativo (Sez. 5.4).

8.2 Cosa mi ha sorpreso

- **Le due ipotesi smentite su Acrobot**: un numero maggiore di bin produce risultati *migliori* (non peggiori, poiché l'aliasing di una griglia grossolana domina la curse of dimensionality), e *meno* esplorazione risulta preferibile a ε costante, salvo poi essere battuta dall'annealing. Il problema non risiedeva nella quantità di esplorazione, ma nel momento in cui viene impiegata (Sez. 5.6, 5.7).
- **SARSA batte Q-Learning su CartPole**, con varianza inferiore. L'aspettativa era opposta; tuttavia la curva misura la performance *con l'esplorazione attiva*, e SARSA è l'unico dei due algoritmi che ne tiene esplicitamente conto nel target.
- **Il bias di TD(0) non è una costante**: sulla policy casuale (episodi corti) TD raggiunge MC, mentre su quella appresa ($\approx$ 330 passi) resta indietro del 13%. La distorsione dipende dalla distanza temporale dal reward, un aspetto che il *random walk* del corso non rende visibile.
- **Senza target network l'apprendimento diventa una lotteria**: l'aspettativa era di osservare un agente "più lento", mentre nella realtà 2 seed su 3 non decollano affatto e solo uno apprende. La media (-20) nasconde il fenomeno, che l'analisi per-seed rivela: la target network non si limita ad accelerare l'apprendimento, lo rende *affidabile* (Sez. 5.9).

8.3 Limiti degli algoritmi

- **Discretizzazione**: rappresenta il collo di bottiglia dominante, con due aspetti contrapposti. Pochi bin non riescono a distinguere stati che richiedono azioni diverse, mentre troppi bin fanno esplodere lo spazio degli stati. Nessuna scelta del numero di bin risolve entrambi i problemi simultaneamente: è necessario un approssimatore di funzione, il che costituisce la motivazione empirica del DQN.
- **Efficienza campionaria e convergenza**: migliaia di episodi per risultati modesti (in assenza di generalizzazione tra stati simili), e con α ed ε costanti la stima fluttua senza mai convergere in modo definitivo.
- **Sparsità del reward**: su Acrobot il segnale (-1 per passo) non indica *quale* azione fosse sbagliata; il credit assignment su 500 passi costituisce il problema reale, e nessuno dei quattro algoritmi tabellari lo affronta in modo diretto.

- **Sensibilità agli iperparametri**: marcata, e con modalità che cambiano da ambiente ad ambiente e persino da algoritmo ad algoritmo (un α elevato danneggia SARSA ma non Q-Learning).
- **Costo del Deep RL**: il DQN risolve il problema della rappresentazione, ma 200k step corrispondono allo 0.4% del budget del paper Nature. È per questo che Pong si arresta a ≈ -13 e che senza target network metà dei seed non decolla.